SC1001: Introduction to Computational Thinking

Week 6: Basics of Functions

Jordan Boyd-Graber

Nanyang Technological University

Slides

Slides and updates will be posted at

`boydgraber.org/teaching/SC_1001/`

Overview

1. Write repetitive code to apply discounts to several items
2. Identify what is repetitive in that code
3. Refactor the logic into a function
4. Compare the function version with the repeated-code version
5. Extend the program to track totals
6. Use a loop to remove hardcoded items

Q1: Discounted Prices

Write Python code to calculate discounted prices for different items.

Each item has an original price and discount rate. Compute

- discount amount
- final price

for several hardcoded items.

How to Read a Format String

```
new = 1 / 3 + 5
print(f"{new:.2f}")  # 5.33
```

- The `f` before the opening quote marks a formatted string (an f-string).

How to Read a Format String

```
new = 1 / 3 + 5
print(f"{new:.2f}")  # 5.33
```

- The `f` before the opening quote marks a formatted string (an f-string).
- Braces insert the value of `new`; the colon starts its format specification.

How to Read a Format String

```
new = 1 / 3 + 5
print(f"{new:.2f}")  # 5.33
```

- The `f` before the opening quote marks a formatted string (an f-string).
- Braces insert the value of `new`; the colon starts its format specification.
- In `.2f`, the final `f` selects fixed-point format, and `.2` sets the precision to 2 digits after the decimal point.

How to Read a Format String

```
new = 1 / 3 + 5
print(f"{new:.2f}")  # 5.33
```

- The `f` before the opening quote marks a formatted string (an f-string).
- Braces insert the value of `new`; the colon starts its format specification.
- In `.2f`, the final `f` selects fixed-point format, and `.2` sets the precision to 2 digits after the decimal point.
- With a precision but no type, as in `.2`, a `float` uses significant digits instead.

Why Format Strings Help

```
new = 1 / 3 + 5
print(new)          # 5.333333333333333
print(f"{new:.2}")  # 5.3: 2 significant digits
print(f"{new:.2f}") # 5.33: 2 decimal places

new = 5
print(f"{new:.2f}") # 5.00
print(f"{new}")     # 5: no format specification
```

Use `.2f` for consistent currency output, including trailing zeros.

Q2: Review the Q1 Code

- Do you notice anything repetitive?
- What parts are the same each time?
- What parts are different each time?
- If the logic had a bug, how many places would need to be fixed?
- What if we had 100 items instead of 4?

Q3: Rewrite Q1 Using a Function

Turn the repeated discount logic into a function and then call that function for each item.

Q4: Review the Function Version

- Which version is shorter and easier to read?
- Which version is easier to update if the logic changes?
- Which version is more organized and reusable?
- Which version would be easier to test and debug?

Q5: Improve the Program

Starting from the function version:

- show the discounted price for each item,
- compute the total price to pay,
- compute the total discount received.

Q5+: Get Items from a List

Adding an item should only require another entry in the data. Revise the code to get the items from a list using list comprehensions.

```
items = [("Item 1", 120.00, 0.15), ("Item 2", 59.90, 0.10),
         ("Item 3", 249.50, 0.20), ("Item 4", 19.99, 0.05)]
```

- Write `compute_item_summary` to return the label, original price, discount amount, and final price.
- Use a list comprehension to create one summary per item.
- Print the summaries as a table with SGD and USD prices.

Q6: Use a While Loop Instead

Enhance the program so that it no longer uses hardcoded values.

- repeatedly ask for the item label
- ask for the original price
- ask for the discount rate
- stop when the user enters a special keyword such as `End`

Optional: error checking and case insensitivity.

Word Counts

- You have text in a variable called `document`
- Create a dictionary called `word_count` that has the words as keys and the number of times the word appears in `document` as the value

Word Counts

- You have text in a variable called `document`
- Create a dictionary called `word_count` that has the words as keys and the number of times the word appears in `document` as the value
- Use whitespace tokenization, so punctuation will still be attached to words

Word Counts

- You have text in a variable called `document`
- Create a dictionary called `word_count` that has the words as keys and the number of times the word appears in `document` as the value
- Use whitespace tokenization, so punctuation will still be attached to words
- Ignore case

Word Counts: Make a Function

Refactor the word-count code into `term_frequency(document)`.

- Take one document as the argument.
- Lowercase words, strip punctuation at their edges, and skip empty words.
- Return the dictionary of word counts.
- How can its keys tell us which words occur in this document?

Document frequency

- TF-IDF measures how important a word is in a document relative to a corpus
- The "idf" is inverse document frequency: words appearing in many documents receive less weight
- Given a list of documents `corpus` $= [\text{doc}_1 \ldots]$, compute the `document_frequency`
- The key should be the word, the value should be the number of documents it appears in
- Reuse `term_frequency(document)`: its keys contain each word once